OS LAB ASSIGNMENT-3

GROUP-18:

Vislavath Bindhu (230101112)

Tippana Likitha Reddy (230101105)

Bommagani Rohitha (230101027)

Annadevara Dedeepya Siri (230101011)

Drive link for deliverables:

https://drive.google.com/drive/folders/1yDc67zJ0GcJ6J4vyshaIsAlRnpWUk6vK

Task 1: Implement Kernel Primitives for IPC:- 1) Objective:

The objective of this task is to extend the xv6 operating system with fundamental Inter-Process Communication (IPC) primitives that go beyond its default, limited capabilities. Specifically:

1. Shared Memory Support
    a. Provide a mechanism for multiple processes to directly share a common memory page.
    b. Enable fast and efficient data exchange without copying between processes.
    c. Ensure proper reference counting and safe cleanup when processes detach.
2. Mailbox-Based Messaging
    a. Implement reliable communication channels (mailboxes) identified by keys.
    b. Allow processes to send and receive integer messages with proper synchronization.
    c. Support blocking semantics so that a sender waits if the mailbox is full, and a receiver waits if the mailbox is empty.

By achieving these, xv6 will be able to support both data sharing (via shared memory) and synchronization/coordination (via mailboxes), enabling more complex multi-process applications and coordination patterns.

2) Code Changes and Expectations:

2.1. File: user/user.h

```
55 int shm_create(int key);
56 void* shm_get(int key);
57 int shm_close(int key);
```

```
59 int mbox_create(int key);
60 int mbox_send(int mbox_id, int msg);
61 int mbox_recv(int mbox_id, int *msg);
```

Without these, user programs (shmtest.c, mboxtest.c) wouldn’t compile because the compiler wouldn’t know about the functions. 2.2.File:

user/usys.pl

```
47 entry("shm_create");
48 entry("shm_get");
49 entry("shm_close");
50 entry("mbox_create");
51 entry("mbox_send");
52 entry("mbox_recv");
```

- We add in usys.pl so that xv6 can auto-generate the assembly stubs (usys.S) for these new system calls.
- Without these entries, user programs could not directly call shm_create() or mbox_create() or other functions , because the kernel wouldn’t know to expose them as system calls.

2.3. File: kernel/sysproc.c

```
114 uint64 sys_shm_create(void) {
115   int key;
116   argint(0, &key);   // Retrieve 1st integer argument into key
117   return (uint64)shm_create(key);
118 }
119
120 uint64 sys_shm_get(void) {
121   int key;
122   argint(0, &key);
123   return (uint64)shm_get(key);
124 }
125
126 uint64 sys_shm_close(void){
127   int key;
128   argint(0, &key);
129   return (uint64)shm_close(key);
130 }
```

```
132 uint64 sys_mbox_create(void) {
133   int key;
134   argint(0, &key);
135
136   return (uint64)mbox_create(key);
137 }
138
139 uint64 sys_mbox_send(void) {
140   int mbox_id, msg;
141   argint(0, &mbox_id);   // first arg
142   argint(1, &msg);       // second arg
143   return (uint64)mbox_send(mbox_id, msg);
144 }
145
146 uint64 sys_mbox_recv(void) {
147   int mbox_id;
148   uint64 msg_addr;
149   argint(0, &mbox_id);    // first arg integer
150   argaddr(1, &msg_addr);  // 2nd arg pointer address
151   return (uint64)mbox_recv(mbox_id, msg_addr);
152 }
```

The six functions are system call handler functions added in sysproc.c. They act as an interface between user-level system call requests and the actual kernel functions that implement shared memory.

1. Fetching Arguments
   a. argint(0, &key); retrieves the first argument (an integer key) passed from the user program and stores it in the kernel variable key. b. This ensures safe transfer of data from user space to kernel space.
2. Calling Kernel Implementation
   a. Each handler then calls the respective kernel function (shm_create, shm_get, or shm_close) with the extracted key.

Returning Result

- The return value from the kernel function is cast to uint64 and returned back to the user program.
- This allows the user process to know whether the call was successful (positive id) or failed (negative value). 2.4.File: Syscall.h

```
25 #define SYS_shm_create 22
26 #define SYS_shm_get    23
27 #define SYS_shm_close  24
28
29
30 #define SYS_mbox_create 25
31 #define SYS_mbox_send   26
32 #define SYS_mbox_recv   27
```

- These lines are in syscall.h, where xv6 assigns a unique number (ID) to every system call. 2.5. File: Syscall.c

```
109 extern uint64 sys_shm_create(void);
110 extern uint64 sys_shm_get(void);
111 extern uint64 sys_shm_close(void);
112 extern uint64 sys_mbox_create(void);
113 extern uint64 sys_mbox_send(void);
114 extern uint64 sys_mbox_recv(void);
```

- This line is a forward declaration (prototype) that tells the compiler:
- “There exists these function names somewhere else in the kernel code, returning a uint64.”

```
147 [SYS_shm_create] sys_shm_create,
148 [SYS_shm_get]    sys_shm_get,
149 [SYS_shm_close]  sys_shm_close,
150
151 [SYS_mbox_create] sys_mbox_create,
152 [SYS_mbox_send]   sys_mbox_send,
153 [SYS_mbox_recv]   sys_mbox_recv,
```

- This line is part of the system call dispatch table (syscalls[]) in xv6.
- It creates a mapping between the system call number (SYS_shm_create, defined in syscall.h) and the actual kernel function (sys_shm_create) that should run when a process calls shm_create() in user space.

2.6.File:Defs.h:

```c
void shm_init(void);
int shm_create(int key);
uint64 shm_get(int key);
int shm_close(int key);

void mbox_init(void);
int mbox_create(int key);
int mbox_send(int mbox_id, int msg);
int mbox_recv(int mbox_id, uint64 msg_addr);
```

- Kernel-wide header needs function prototypes for new IPC functions so other kernel files can call them. 2.7.File:main.c

```c
    shm_init();
    mbox_init();
```

- Kernel subsystems need initialization at boot.
- This sets up global tables and locks before any process uses IPC. 2.8.File:vm.c

```c
void
freewalk(pagetable_t pagetable)
{
  // there are 2^9 = 512 PTEs in a page table.
  for(int i = 0; i < 512; i++){
    pte_t pte = pagetable[i];
    if((pte & PTE_V) && (pte & (PTE_R|PTE_W|PTE_X)) == 0){
      // this PTE points to a lower-level page table.
      uint64 child = PTE2PA(pte);
      freewalk((pagetable_t)child);
      pagetable[i] = 0;
    } else if(pte & PTE_V){
      uint64 pa = PTE2PA(pte);
      // skip freeing shared pages
      if(!is_shared_physical((char*)pa)){
        kfree((void*)pa);
      }
      pagetable[i] = 0;
    }
  }
  kfree((void*)pagetable);
}
```

-
- freewalk() normally frees all physical pages on process exit.
- Shared memory pages must not be freed until last user closes them.
- Prevents accidental freeing of shared pages while still in use by other processes.

```c
extern int is_shared_physical(char *pa);
```

- This is added in vm.c so the memory manager can identify shared pages and avoid freeing them prematurely, ensuring correct shared memory behavior.

2.9.File:Makefile

```
        $U/_shmtest\
        $U/_mboxtest\
```

- Ensures shmtest.c,mboxtest.c are compiled and included in the file system image during make.

```
  $K/shm.o \
  $K/mbox.o
```

- In xv6, the Makefile (or kernel/Makefile) lists all the object files (.o) that must be compiled and linked together to build the kernel. 2.10:File:kernel/shm.c

This file implements shared memory support in xv6, allowing processes to map and share physical pages.

1. Shared Memory Region Table:

- The kernel maintains a global table (shm_table) of up to 16 shared memory regions.
- Each region is identified by a key.
- page stores the physical memory address returned by kalloc().
- refcnt counts how many processes are currently mapping this region.
- is_shared marks that the entry is being used for shared memory.
- Access is synchronized with a spinlock to avoid race conditions.

2. Initialization:shm_init()

- Runs at boot time.
- Sets all regions to “empty” (key = -1).
- Prepares spinlock for synchronization.

3. Creating Shared Memory (shm_create)

- If the key already exists → reuse it.
- Otherwise, allocate a new physical page with kalloc().
- Assign the key and mark the entry as used.
- Return the index in the shared memory table (acts like a handle).

4. Mapping Shared Memory (shm_get)

- Finds the region with the given key.
- Assigns a fixed virtual address for each region slot:ensures all processes see the same virtual address for the same shared region.
- If not already mapped in this process, use mappages to map the physical page into the process’s page table.
- Increments reference count.

5. Closing Shared Memory (shm_close)

- Decrements refcnt when a process is done.
- If the last process detaches (refcnt == 0), free the physical page with kfree().
- Clear the region entry so it can be reused.

6. Check If a Page is Shared (is_shared_physical)

- Helper function: check if a given physical address belongs to a shared memory region.
- Useful in the kernel (e.g., in vm.c) to avoid accidentally freeing shared memory pages too early.

2.11:user/shmtest.c

This program demonstrates inter-process communication via shared memory in xv6.
It shows that multiple processes can map the same physical page, read from it, and write to it, with changes visible to all.

1. Create and Get Shared Memory The parent process creates a shared

memory region with key 42.

- shm_get(42) maps the shared page into its address space and returns a user virtual address ptr.
- Parent now has access to this memory.

2. Fork a Child Process

- A child process is created.
- Both parent and child can use the same shared memory key = 42.

3. Child Process Behavior

- Child attaches the same shared memory page (shm_get(key)).
- Writes "hello" into it.
- Prints what it wrote.
- Closes its reference (shm_close), but memory still exists because the parent is using it.

4. Parent After Child Finishes Parent waits until child finishes.

- Then reads the shared memory and sees "hello" (child’s update).
- Parent overwrites it with "world".

5. Grandchild Verification Parent forks again to create a grandchild.

- Grandchild attaches to the same shared memory (shm_get(42)).
- It sees "world", confirming that updates by the parent are visible.
- Parent prints a PASS message to confirm shared memory works.

Output:

```
$ shmtest
shm_create: created key 42 at index 0
shm_get: mapped pid 3 key 42
[shmtest] parent got ptr 0x0000000000004000
shm_get: mapped pid 4 key 42
[shmtest] child  got ptr 0x0000000000004000
[shmtest] child saw: "hello"
[shmtest] parent now sees: "hello"
[shmtest] parent wrote: "world"
shm_get: mapped pid 5 key 42
[shmtest] grandchild saw: "world"
[shmtest] PASS: shared page is the same and updates are visible across processes
```

2.12:File:kernel/mbox.c

This file implements a mailbox-based Inter-Process Communication (IPC) mechanism in the xv6 kernel. Mailboxes allow processes to send and receive integer messages in a synchronized manner.

1. Data Structure: struct mailbox

- Each mailbox is a bounded buffer with size MBOX_SIZE (here = 1).
- Uses head-tail pointers for circular queue behavior.
- Spinlock ensures atomic access when multiple processes use the mailbox.

2. Initialization: mbox_init()

- Initializes all mailboxes.
- key = -1 means unused.
- Lock is set up for each mailbox.

3. Creating a Mailbox: mbox_create(int key)

- If a mailbox with given key exists, returns its index.
- Otherwise creates a new mailbox if free.
- Returns -1 if all MAX_MBOX are used.

4. Sending a Message: mbox_send(int mbox_id, int msg)

- Locks the mailbox.
- If buffer is full, process sleeps until space is available.
- Inserts message, updates tail and count.
- Calls wakeup so waiting receivers can continue.

5. Receiving a Message: mbox_recv(int mbox_id, uint64 msg_addr)

- Locks the mailbox.
- If buffer is empty, process sleeps until a sender puts data.
- Retrieves the message, updates head and count.
- Calls wakeup so senders can continue.
- Uses copyout() to safely copy the message into user space.

2.13.File:user/mboxtest.c

This program demonstrates message passing between parent and child processes using your mailbox API. 1. Mailbox

Creation

- Creates or finds a mailbox with key 42.
- Returns a mailbox ID (id) that will be used for sending/receiving messages.
- If it fails, the program exits.

2. Fork a Child Process □ Creates a

child process.

- Both parent and child will share the mailbox identified by id.
- From this point:
- Child (pid == 0) will receive messages. □ Parent (pid > 0) will send messages.

3. Receiver (Child Process) Child

waits for 5 messages.

- Calls mbox_recv(id, &x) each time:
- Blocks if the mailbox is empty.
- When a message arrives, stores it in variable x.
- Prints what it received.
- Exits after receiving all messages.

4. Sender (Parent Process) Parent

sends 5 messages (10, 20, 30, 40,

50) into the mailbox.

- Each mbox_send(id, i*10) blocks if the mailbox is full.
- After sending all messages, parent waits for the child to finish.
- Prints done and exits.

Output:

```
$ mboxtest
[mboxtest] parent sent 10
[mboxtest] child received 10
[mboxtest] parent sent 20
[mboxtest] child received 20
[mboxtest] parent sent 30
[mboxtest] child received 30
[mboxtest] parent sent 40
[mboxtest] child received 40
[mboxtest] parent sent 50
[mboxtest] child received 50
[mboxtest] done
```

Task 2: The Intertwined Memory Challenge

1. Overview

This project implements a two-process traversal of a maze using shared memory and mailbox-based communication in xv6. The system consists of:

- A master program that sets up the maze in shared memory, creates mailboxes, and spawns two child processes.
- A process program that executes the traversal, communicating with its peer using the mailboxes.

The goal is to demonstrate inter-process communication (IPC) using shared memory and mailboxes while avoiding deadlock.

2.MAKEFILE changes

```
149         $U/_process\
150         $U/_master\
```

 Ensures master.c ,process.c are compiled and included in the file system image during make.(even though process is not run explicitly in terminal)

3. Path Structure in Shared Memory

- A single shared memory page is created once by the master process using shm_create(KEY). All other processes then attach to the same page using shm_get(KEY).
- The maze is stored as an integer array (shared_mem[]).
  Each index represents a “cell” in the maze, and the value at that index encodes the next position the other process should move to. This creates an intertwined path for two processes (A and B).

Example path used in this project:

- shared_mem[0] = 1 → when A is at 0, B should go to 1.

- shared_mem[1] = 2 → when B is at 1, A should go to 2.
- shared_mem[2] = 3 → when A is at 2, B should go to 3.
- … continues alternating …
- shared_mem[7] = 0xFFFF → special end marker indicating termination.

shm_create(KEY) (used in Master only)

- Allocates or registers a shared memory page with identifier KEY.
- Returns a non-negative handle on success, negative on failure.
- Must be called before any child process tries to attach.
    - In our program:
        master calls shm_create(KEY) once, then fills shared_mem[] with the maze structure.

shm_get(KEY) (used in Master + Children)

- Maps the shared page identified by KEY into the address space of the calling process.
- Returns a pointer (e.g., (int*)) to the shared memory region.
- On failure returns an error sentinel (commonly (void*)-1 or 0).
    - In our program:
- The master calls shm_get(KEY) immediately after creation to initialize the maze array.
- Each child process calls shm_get(KEY) after exec() to attach to the same page.

Visibility & Synchronization

- The shared page is physically the same for all processes.
- Any update by one process is immediately visible to the others.
- To prevent race conditions, we do not read/write blindly; instead, we use the mailbox protocol (mbox_send, mbox_recv) to coordinate whose turn it is to read/write. o This ensures correctness and prevents deadlock.

4. Communication Protocol

To ensure both processes (A and B) remain in lockstep while traversing the maze, we use two unidirectional mailboxes for message passing:

- mbox_A_to_B → carries messages from Process A to Process B
- mbox_B_to_A → carries messages from Process B to Process A

Step-by-Step Protocol 1. Send current position

Each process first communicates its current maze index to the peer using its outbound mailbox.

2. Receive peer's position

It then blocks on its inbound mailbox to receive the other process's position.

3. Look up next move

The process uses the peer's position as an index into shared_mem[] to determine its own next position.

Why Two Mailboxes?

- A single bidirectional mailbox could lead to ordering ambiguity (both processes trying to read or write at the same time).
- Two mailboxes provide clear directionality, so each process always knows where to send and where to receive from. Effect of the Protocol

This protocol guarantees:

- Synchronization: Neither process moves ahead without the other.
- Correctness: Each move is computed based on the peer's reported position.
- Determinism: The traversal proceeds in the exact intended sequence of alternating steps through the maze.

5. Deadlock Avoidance Strategy

If both processes attempted to send first, they would both block waiting for the other to receive, causing a deadlock. To prevent this:

- Process A: send first, then receive.
 Process B: receive first, then send.

This complementary strategy ensures:

- When A sends, B is waiting to receive. When B sends, A is waiting to receive.

6. Execution Flow

1. Master Process Initialization

   - Calls shm_create(KEY) to allocate the shared memory page.
   - Calls shm_get(KEY) to map it and writes the maze structure (path indices and end marker).

2. Mailbox Creation

   - Creates two mailboxes:
     - mbox_a_to_b (for messages from A to B).
       mbox_b_to_a (for messages from B to A).

3. Forking Child Processes

   - Forks Process A (starts at position 0, role = send-first).
   - Forks Process B (starts at position 1, role = receive-first).

4. Maze Traversal o At each step:

     - One process sends its current position via its send mailbox.
     - It receives the other's position via the recv mailbox.
     - It uses the received position as an index into shared_mem[] to find the next move.

   - This continues alternately, ensuring synchronization.

5. Termination

   - When a process looks up END_MARKER (0xFFFF) in shared memory, it terminates and reports completion.

6. Master Completion

- After both children exit, the master process prints:
  - master: maze run complete

7.OUTPUT:

```
$ master
shm_create: created key 42 at index 0
shm_get: mapped pid 5 key 42
shm_get: mapped pid 6 key 42
shm_get: mapped pid 7 key 42
B at 1, received 0 from A, next 3
A at 0, received 1 from B, next 2
B at 3, received 2 from A, next 5
A at 2, received 3 from B, next 4
B at 5, received 4 from A, next 7
A at 4, received 5 from B, next 6
B at 7, received 6 from A, next 65535
B has reached the end of its path!
A at 6, received 7 from B, next 65535
A has reached the end of its path!
master: maze run complete
```

This log demonstrates:

- Alternating movement between A and B.
- Correct traversal of the intertwined path.
- Proper termination without deadlock.